University of Asia Pacific
Department of Computer Science and Engineering
REPORT: 05

Course Code: CSE 402
Course Title: Operating System Lab

Submitted by:

Name: Rajash Majumdar

Student ID:23101144

Semester : 4th Year 1st Semester

Section: C2

Submitted to:

Atia Rahman Orthi,
Lecturer
University of Asia Pacific

Problem Statement:

Implement Preemptive SJF CPU Scheduling for the following scenario, where No. of processes = 5, and the process information is:

P id+	AT	BT
P1	4	2
P2	2	2
P3	1	3
P4	0	6
P5	3	1

Use Time Quantum (TQ) = 2. Calculate CT, TAT, WT, Average TAT, and Average WT for all processes. Also represent the process execution sequence.

Why is Preemptive SJF Used?

Preemptive SJF, also known as Shortest Remaining Time First (SRTF), gives priority to the process with the shortest remaining execution time.
Unlike non-preemptive SJF, a running process can be interrupted when another suitable process becomes available. This can help reduce the average waiting time and turnaround time, especially when short processes arrive while a longer process is running.

Result:

PID	AT	BT	CT	TAT	WT
P1	4	2	7	3	1
P2	2	2	4	2	0
P3	1	3	10	9	6
P4	0	6	14	14	8
P5	3	1	5	2	1

Average TAT = 6.00
Average WT = 3.20

Execution Sequence:

P4 → P2 → P5 → P1 → P3 → P4 → P3 → P4

Discussion:

In this experiment, SJF Preemptive scheduling was implemented using Python with a time quantum of 2. The program selects the process with the shortest remaining burst time from the available processes. A process may be interrupted when another process has a shorter remaining time. The main difficulty was tracking the remaining burst time and calculating the correct completion time when a process executes multiple times. The obtained Average TAT is 6.00 and Average WT is 3.20.

Conclusion:

The experiment successfully implemented SJF Preemptive CPU Scheduling. The program calculated CT, TAT and WT for all five processes. The Average TAT was 6.00 and the Average WT was 3.20. This experiment helped us understand how preemption and shortest remaining time can improve CPU scheduling performance.

Git Repository link :

https://github.com/Rajash144/OS-CPU-Scheduling-Labs